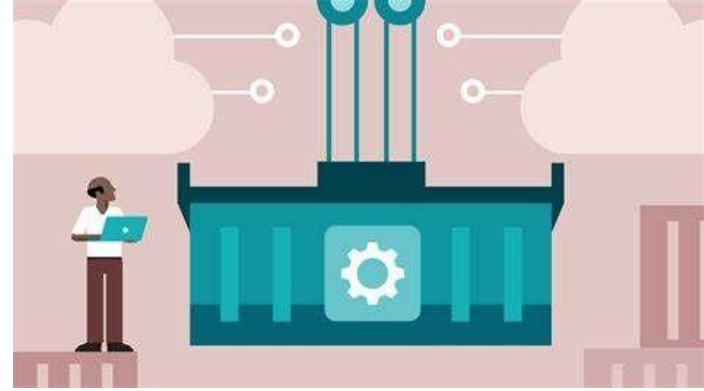
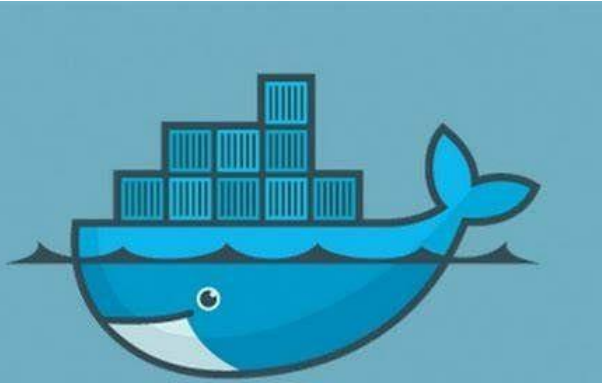




Containers



The Challenge

Do services
and apps
interact
appropriately

Can I migrate
smoothly
and quickly?

 User DB
postgresql + pgv8 +
v8

 Queue  Analytics DB
Redis + redis-sentin +
thrift + h a doop + hive +

 Static
website
nginx 1.5 + modsecurity +
openssl + bootstrap 2

 Background workers
Python 3.0 + celery + pyredis + libcurl +
ffmpeg + libopencv + nodejs +
phantomjs



Web frontend
Ruby + Rails + sass +
Unicorn

 API
endpoint
Python 2.7 + Flask + pyredis + celery +
psycopg + postgresql-client



Public
Cloud

Disaster
recovery
Production
Servers



Production
cluster

Contributor's
laptop



Development VM



QA server

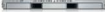
Customer Data
Center



Multiplicity
of Stacks

Multiplicity of
hardware
environments

Results in N X N compatibility nightmare

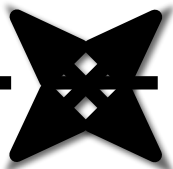
	Static website	?	?	?	?	?	?	?
	Web frontend	?	?	?	?	?	?	?
	Background workers	?	?	?	?	?	?	?
	User DB	?	?	?	?	?	?	?
	Analytics DB	?	?	?	?	?	?	?
	Queue	?	?	?	?	?	?	?
		Development VM	QA Server	Single Prod Server	Onsite Cluster	Public Cloud	Contributor's laptop	Customer Servers
								

Cargo Transport Pre-1960

Multiplicity
of Goods



Do I worry
about how
goods
interact (e.g.
coffee beans
next to
spices)



Multiplicity
of methods
for
transporting/
storing

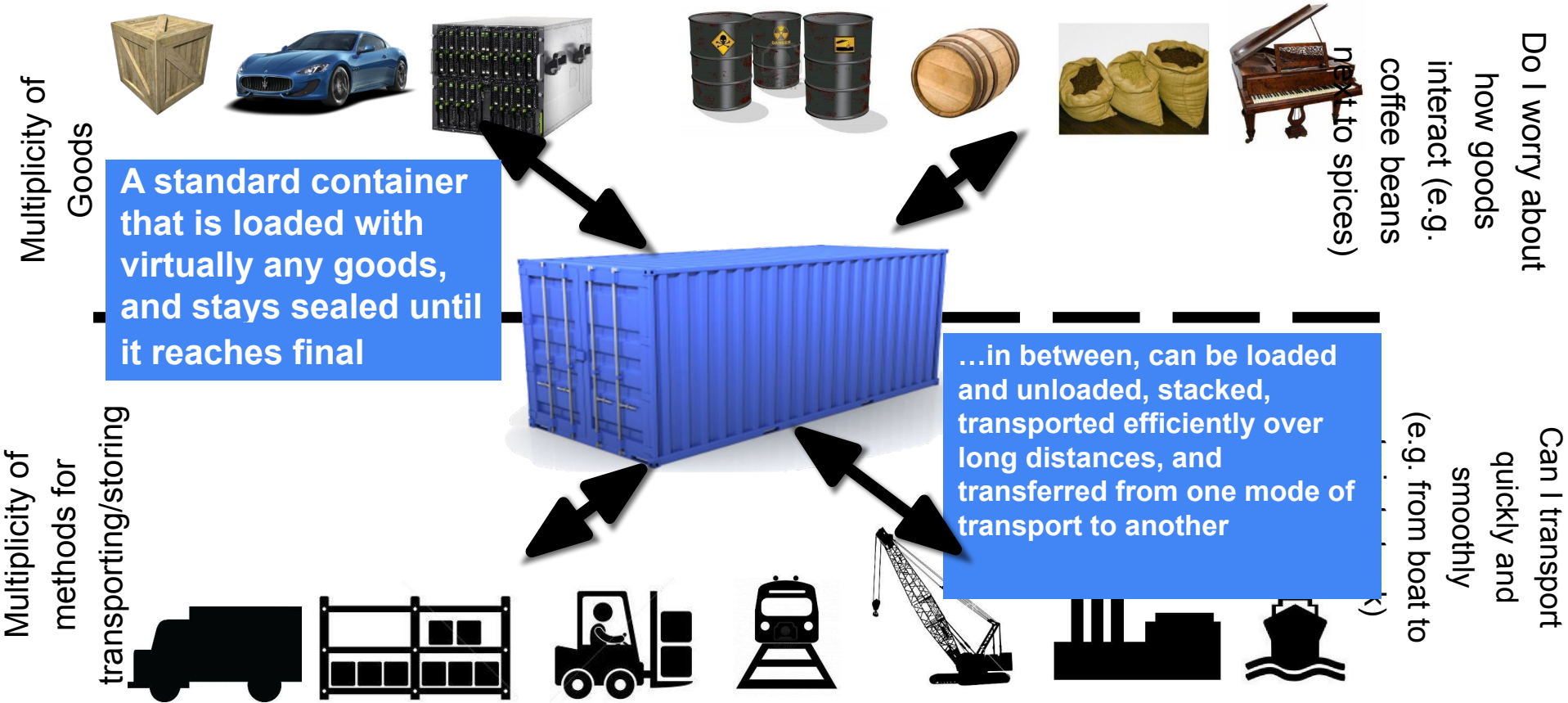


Can I
transport
quickly and
smoothly
(e.g. from boat
to train to
truck)

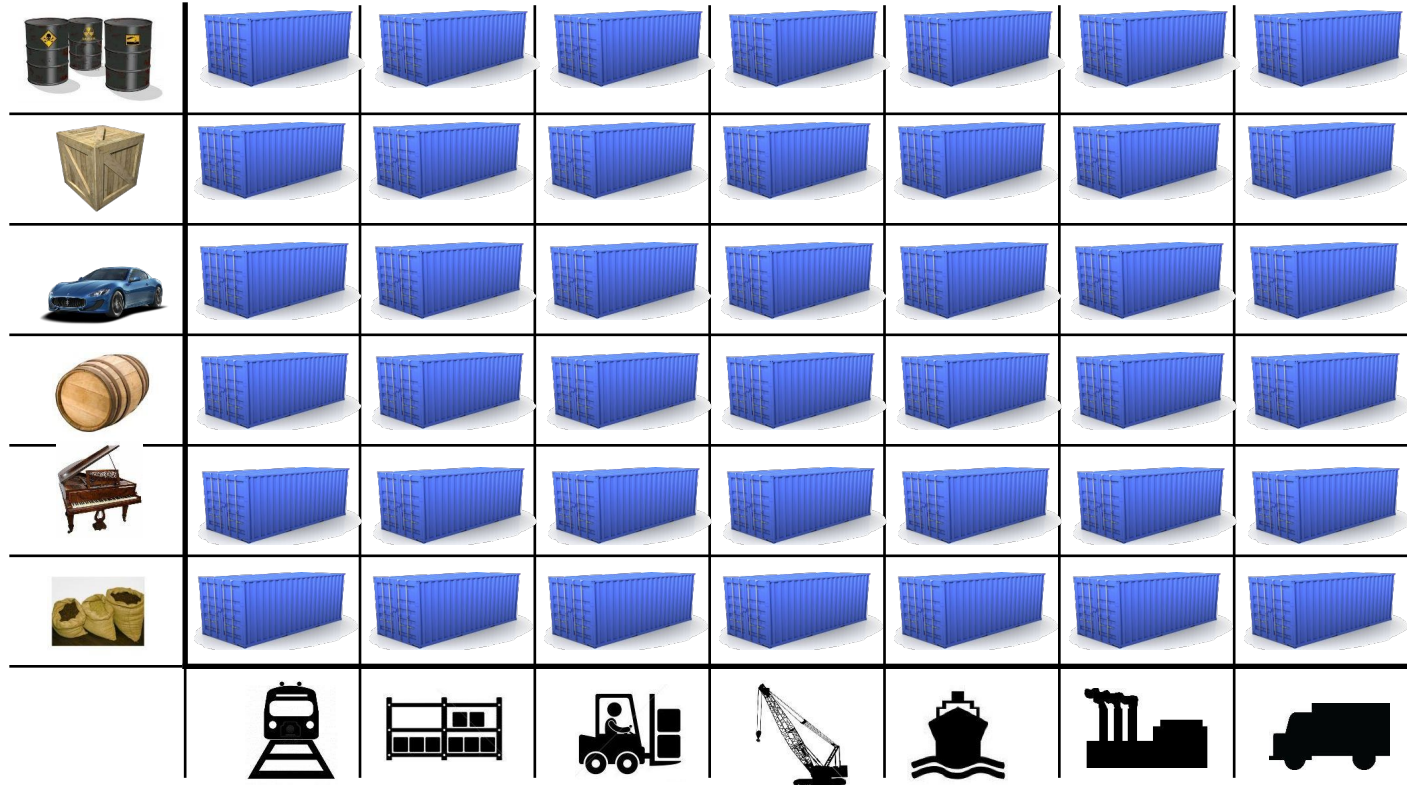
NxN Matrix?

	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
							

Solution: Shipping Container



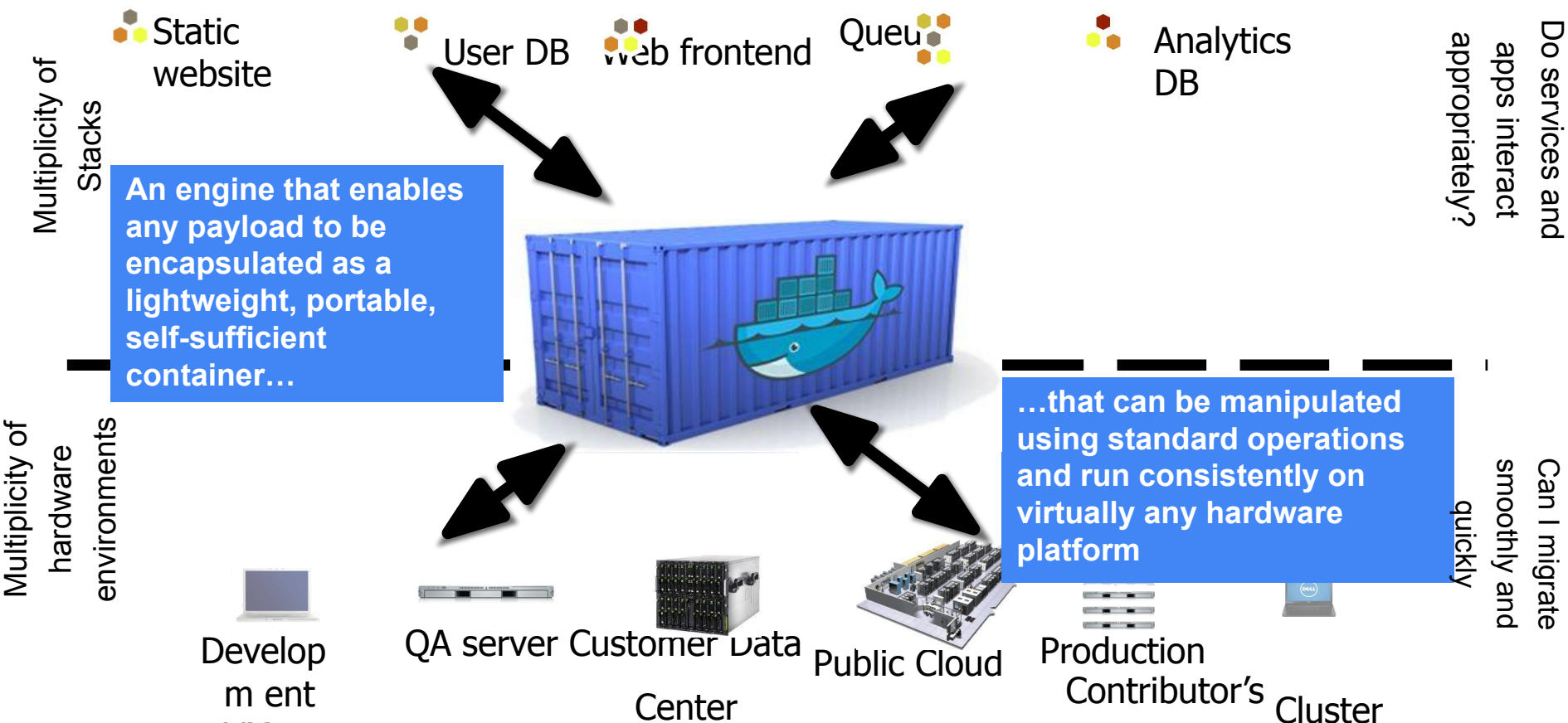
This eliminated the NXN problem...



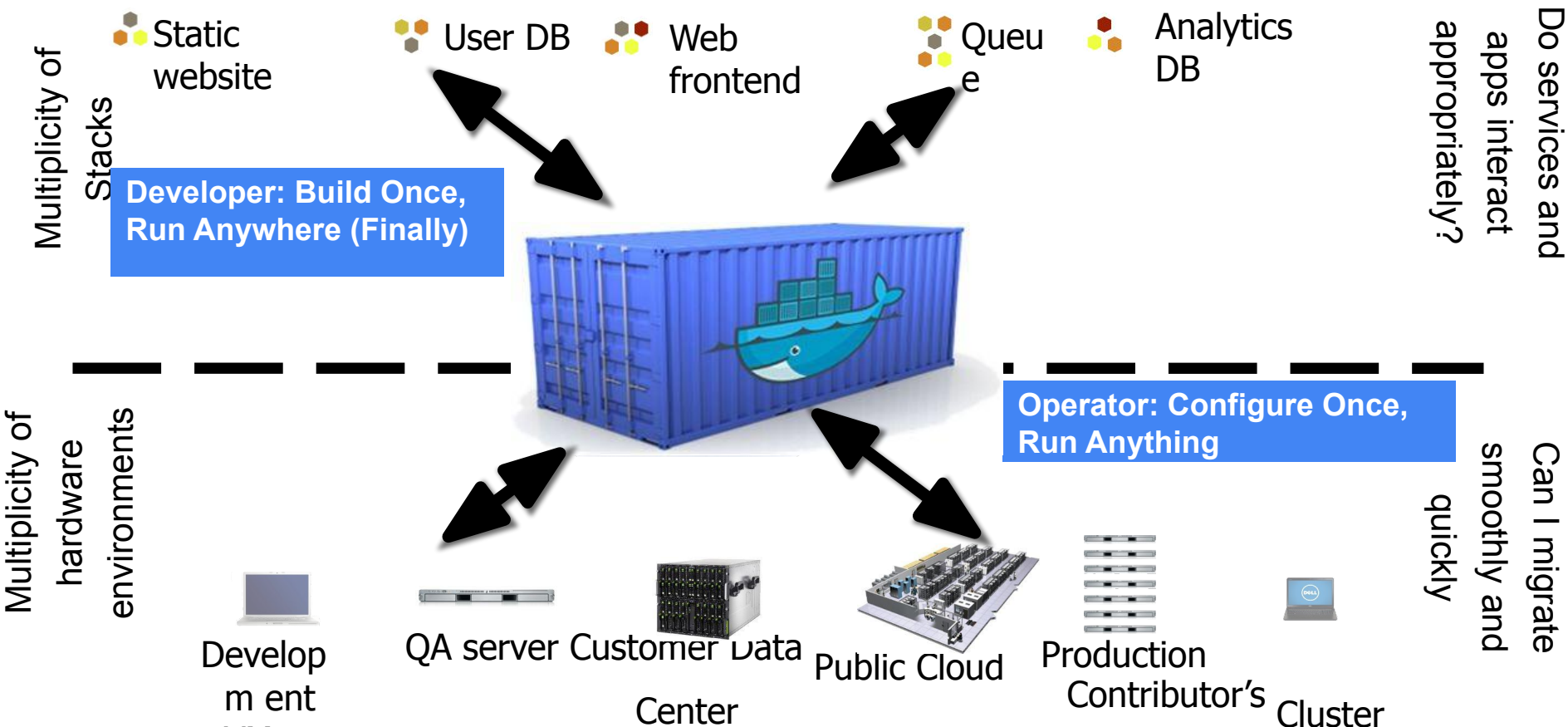


- 90% of all cargo now shipped in a standard container
- Order of magnitude reduction in cost and time to load and unload ships
- Massive reduction in losses due to theft or damage
- Huge reduction in freight cost as percent of final goods (from >25% to <3%)
massive globalizations
- 5000 ships deliver 200M containers per year

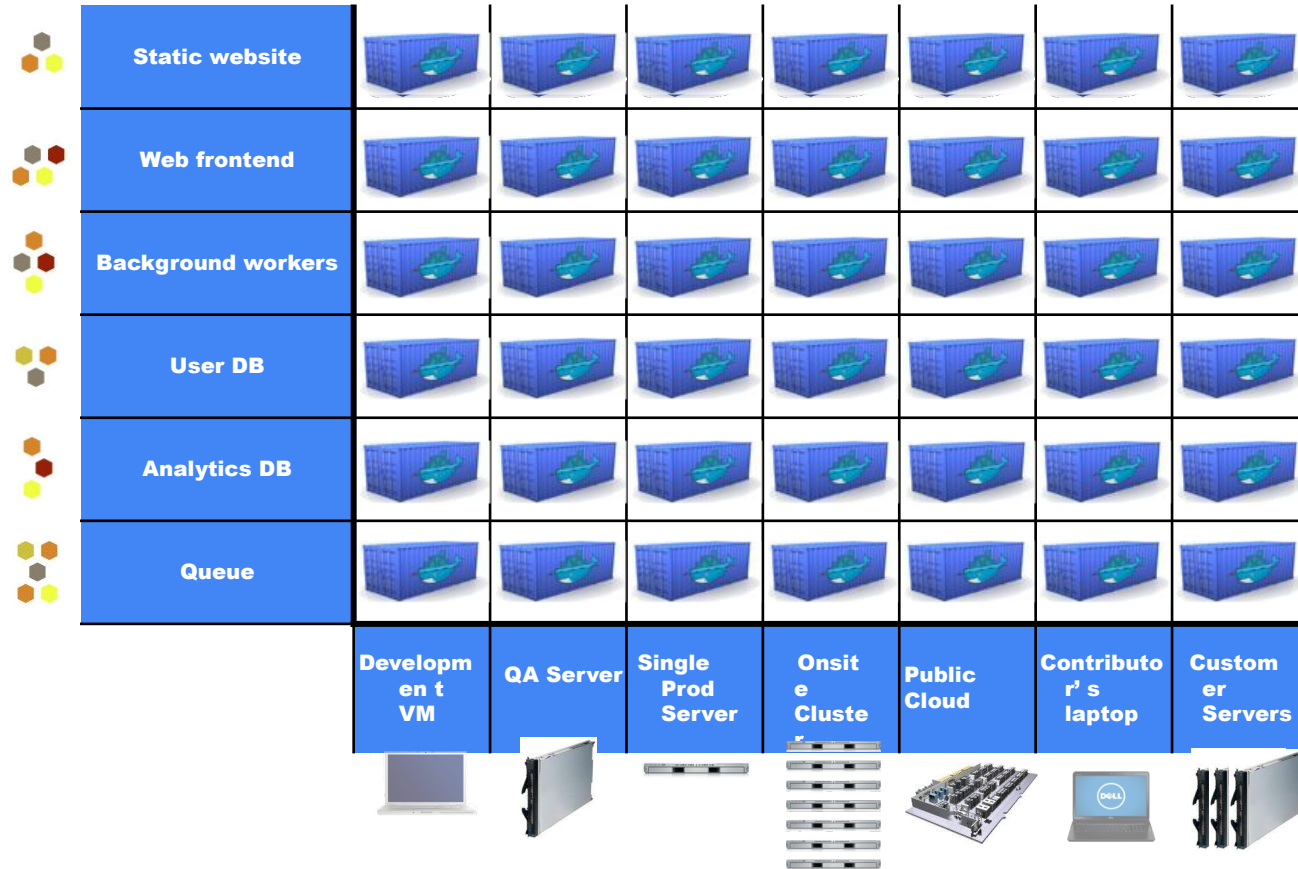
Docker is a shipping container system for code



Or...put more simply



Docker solves the NXN problem



Building Custom Docker Images

- A **Dockerfile** is a script that contains instructions for building a Docker image.
- It is used to automate the process of creating a container image.
- It typically includes information about the **base image**, any **additional software** that needs to be installed, and any **configurations or environment variables** that need to be set.
- The basic syntax for Dockerfile is:
INSTRUCTION arguments

Docker Compose

- **Docker Compose** is a tool for defining and running multi-container Docker applications.
- It is used to define the **services**, **networks** and **volumes for the application**, and then **starts** and **stops** all of the containers with a **single command**.
- The configuration for the application is defined in a **docker-compose.yml** file, which specifies the services, their dependencies, and their configuration.
- This allows developers to easily **manage and scale their applications**, and makes it easy to run the application in different environments.
- It also provides a way to run **multiple container applications** together, by grouping all the services in **one compose file**.

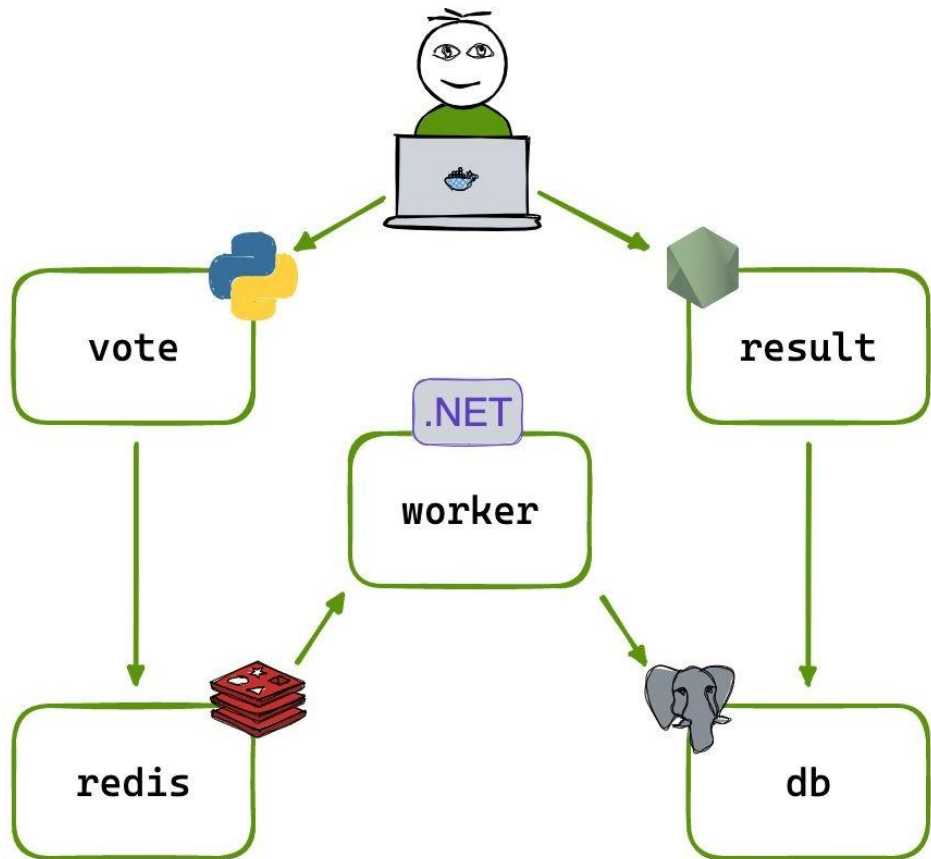
Docker Compose: Installation

Installing Docker Compose:

- `sudo curl -L`
`"https://github.com/docker/compose/releases/download/1.29.2/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose`
- `sudo chmod +x /usr/local/bin/docker-compose`

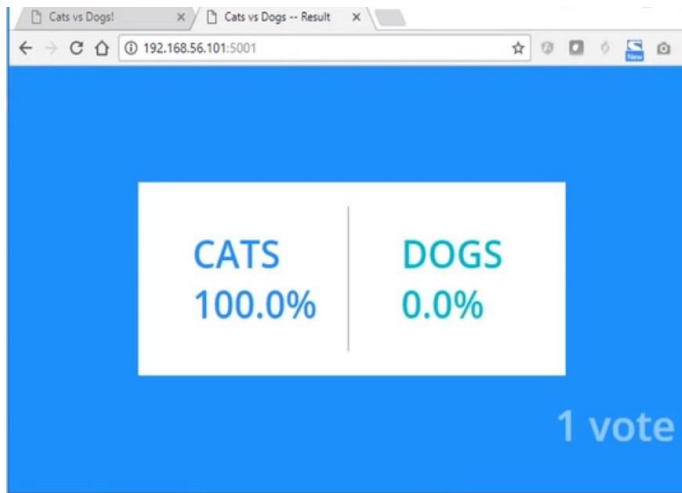
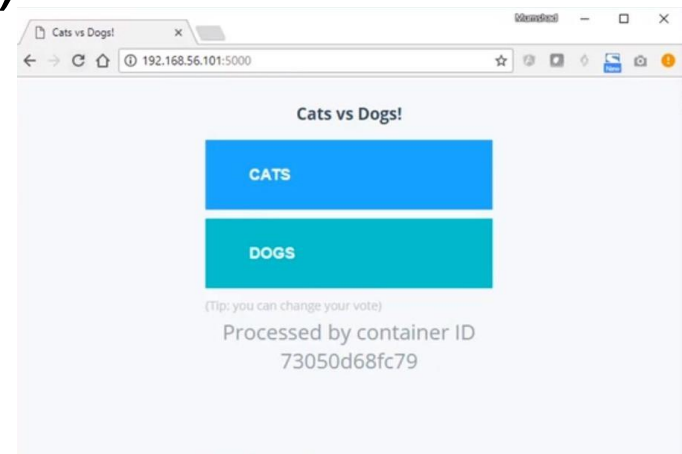
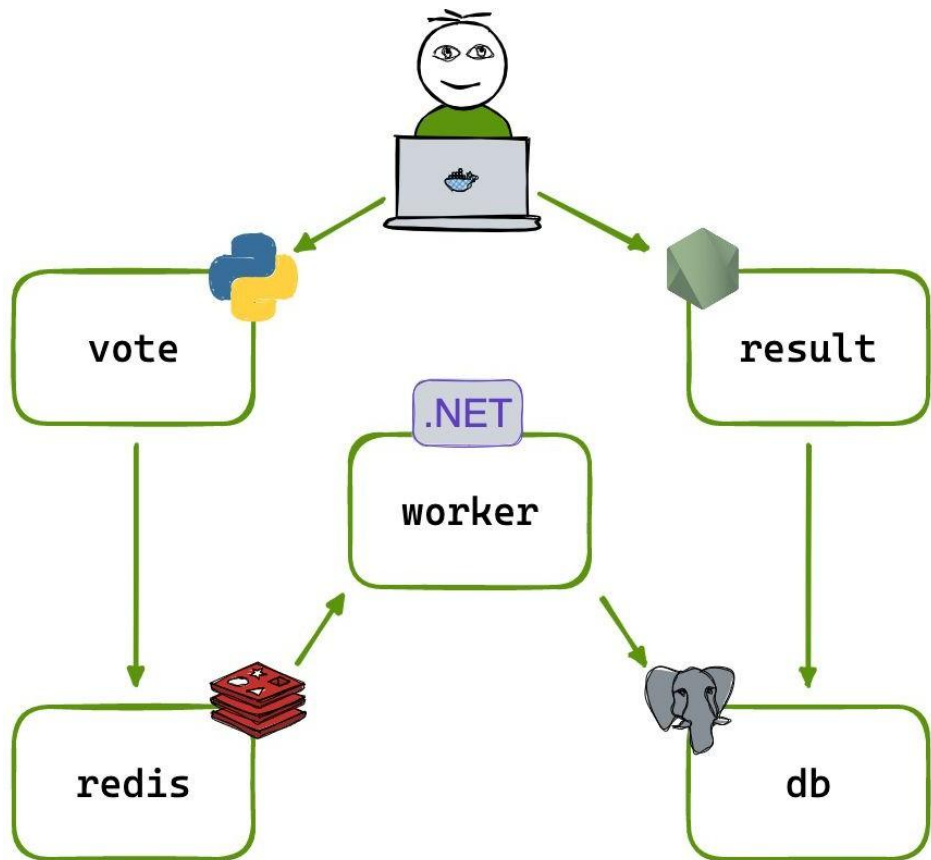
If you are using Docker Desktop, docker-compose is already installed!

Docker Compose: Voting App



- A front-end web app in [Python](#) which lets you vote between two options
- A [Redis](#) which collects new votes
- A [.NET](#) worker which consumes votes and stores them in DB
- A [Postgres](#) database backed by a Docker volume
- A [Node.js](#) web app which shows the results of the voting in real time

Docker Compose: Voting App (Cont.)



Docker Compose: Voting App (Cont.)

```
docker run -d --name=redis redis
```

```
docker run -d --name=db postgres
```

```
docker run -d --name=vote -p 5000:80 voting-app
```

```
docker run -d --name=result -p 5001:80 result-app
```

```
docker run -d --name=worker worker
```

voting-app



result-app



in-memory DB



db

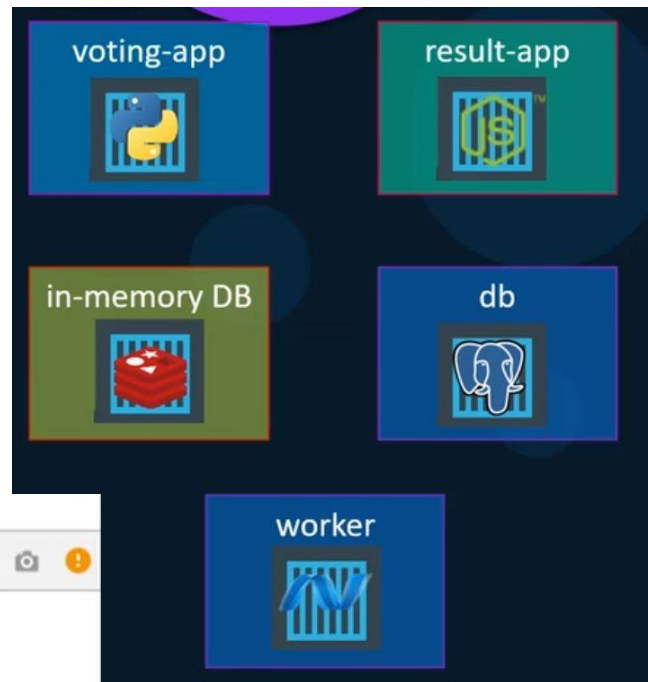
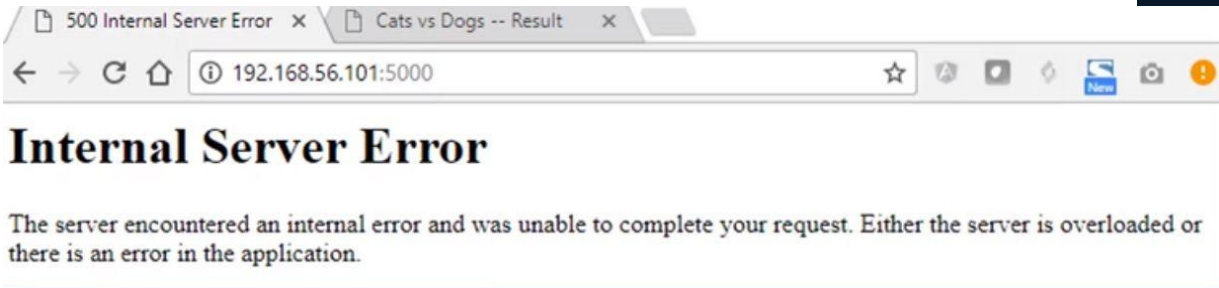


worker



Docker Compose: Voting App (Cont.)

```
docker run -d --name=redis redis
docker run -d --name=db postgres
docker run -d --name=vote -p 5000:80 voting-app
docker run -d --name=result -p 5001:80 result-app
docker run -d --name=worker worker
```



Docker Compose: Voting App (Cont.)

```
docker run -d --name=redis redis
docker run -d --name=db postgres
docker run -d --name=vote -p 5000:80 --link redis:redis voting-app
docker run -d --name=result -p 5001:80 result-app
docker run -d --name=worker worker
```

```
def get_redis():
    if not hasattr(g, 'redis'):
        g.redis = Redis(host="redis", db=0, socket_timeout=5)
    return g.redis
```

```
/app # cat /etc/hosts
127.0.0.1        localhost
::1             localhost ip6-localhost ip6-loopback
fe00::0         ip6-localnet
ff00::0         ip6-mcastprefix
ff02::1         ip6-allnodes
ff02::2         ip6-allrouters
172.17.0.2       redis 89cd8eb563da
```

- The `--link` option in `docker run` allows you to link one container to another. Syntax:
`docker run --link <name/id of container to link>:<alias in the running container> <image name>`
- `--link redis:redis` in the example is linking `voting-app` with `redis` container whereas `redis` is also used alias in `voting-app` to access the `redis` container.

Docker Compose (Cont.)

```
docker run -d --name=redis redis
```

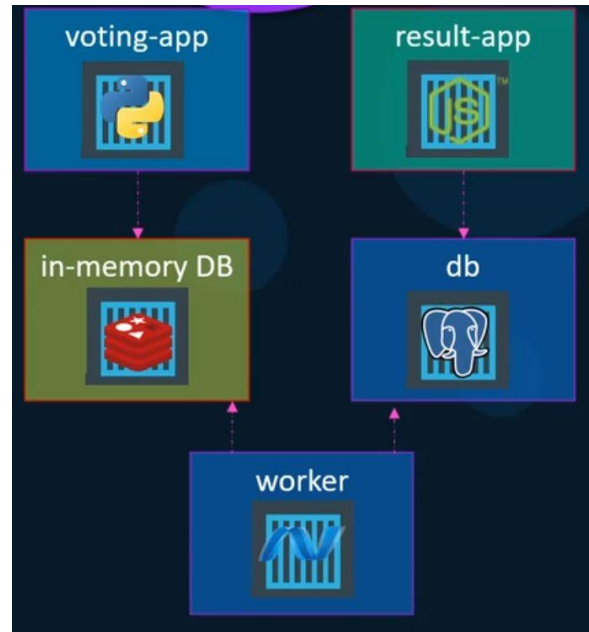
```
docker run -d --name=db postgres
```

```
docker run -d --name=vote -p 5000:80 --link redis:redis voting-app
```

```
docker run -d --name=result -p 5001:80 --link db:db result-app
```

```
docker run -d --name=worker --link db:db --link redis:redis worker
```

```
try {  
    Jedis redis = connectToRedis("redis");  
    Connection dbConn = connectToDB("db");  
  
    System.err.println("Watching vote queue");
```



Docker Compose: Voting App (Cont.)

The image displays two versions of a `docker-compose.yml` file for a voting application, comparing the use of pre-built images versus local builds.

Left File (Pre-built Images):

```
redis:
  image: redis
db:
  image: postgres:9.4
vote:
  image: voting-app
  ports:
    - 5000:80
  links:
    - redis
result:
  image: result
  ports:
    - 5001:80
  links:
    - db
worker:
  image: worker
  links:
    - db
    - redis
```

Right File (Local Builds):

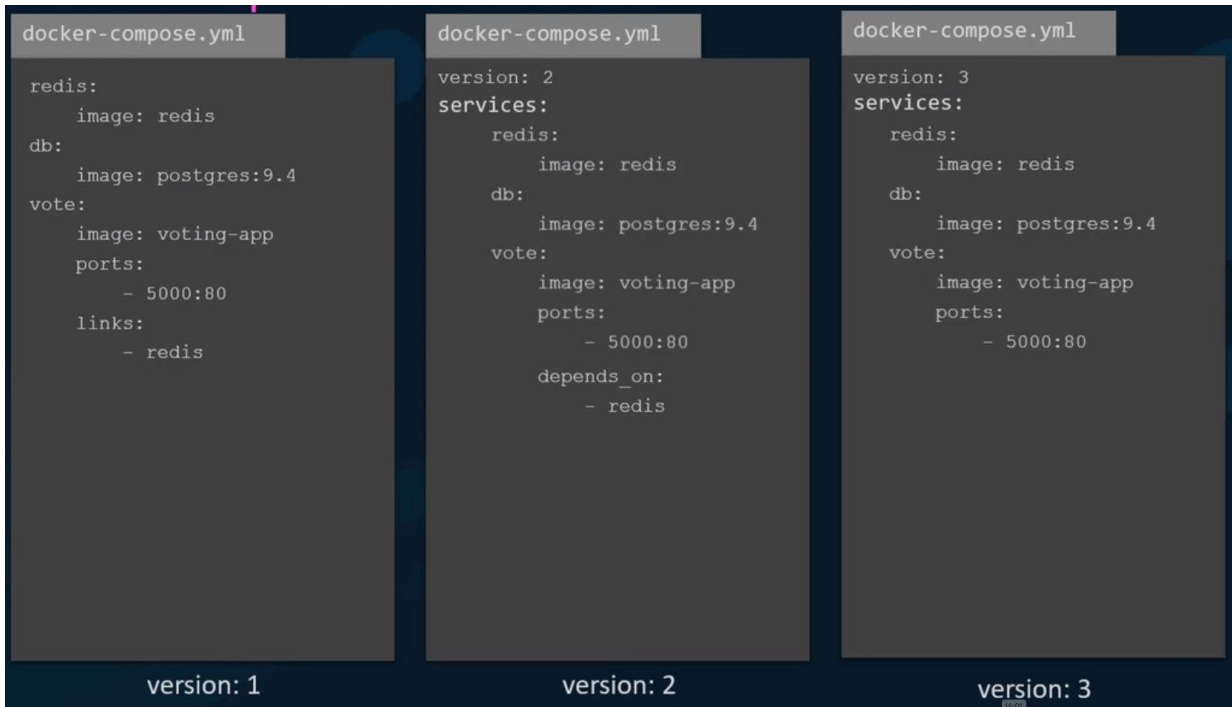
```
redis:
  image: redis
db:
  image: postgres:9.4
vote:
  build: ./vote
  ports:
    - 5000:80
  links:
    - redis
result:
  build: ./result
  ports:
    - 5001:80
  links:
    - db
worker:
  build: ./worker
  links:
    - db
    - redis
```

On the right, a screenshot of the GitHub repository `dockersamples/example-voting-app` is shown, displaying the file structure:

- static/stylesheets
- templates
- Dockerfile
- app.py
- requirements.txt

If the image is not in the registry. We can instruct docker compose to build the image from a local folder which contain source code of the app and Dockerfile

Docker Compose: Voting App (Cont.)



The image displays three panels, each representing a different version of Docker Compose syntax for a Voting App. Each panel has a title bar 'docker-compose.yml' and a code block below it. The panels are labeled 'version: 1', 'version: 2', and 'version: 3' at the bottom.

version: 1

```
redis:
  image: redis
db:
  image: postgres:9.4
vote:
  image: voting-app
  ports:
    - 5000:80
  links:
    - redis
```

version: 2

```
version: 2
services:
  redis:
    image: redis
  db:
    image: postgres:9.4
  vote:
    image: voting-app
    ports:
      - 5000:80
    depends_on:
      - redis
```

version: 3

```
version: 3
services:
  redis:
    image: redis
  db:
    image: postgres:9.4
  vote:
    image: voting-app
    ports:
      - 5000:80
```

The latest is version 3. All the features of version 2 are available in it. We do not need to specify links and it can support multiple networks. We mostly use version 2.

Docker Compose: Voting App (Cont.)

```
docker run -d --name=redis redis
```

```
docker run -d --name=db postgres:9.4
```

```
docker run -d --name=vote -p 5000:80 --link redis:redis voting-app
```

```
docker run -d --name=result -p 5001:80 --link db:db result-app
```

```
docker run -d --name=worker --link db:db --link redis:redis worker
```

db:db = db

```
docker-compose.yml
```

```
redis:
  image: redis
db:
  image: postgres:9.4
vote:
  image: voting-app
  ports:
    - 5000:80
  links:
    - redis
result:
  image: result-app
  ports:
    - 5001:80
  links:
    - db
worker:
  image: worker
  links:
    - redis
    - db
```

First time, you need to install docker-compose. You can do using the following two commands:

- o `sudo curl -L "https://github.com/docker/compose/releases/download/1.29.2/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose`
- o `sudo chmod +x /usr/local/bin/docker-compose`

```
docker-compose up
```

Docker Compose: Voting App (Cont.)

```
docker-compose.yml

version: 2
services:
  redis:
    image: redis

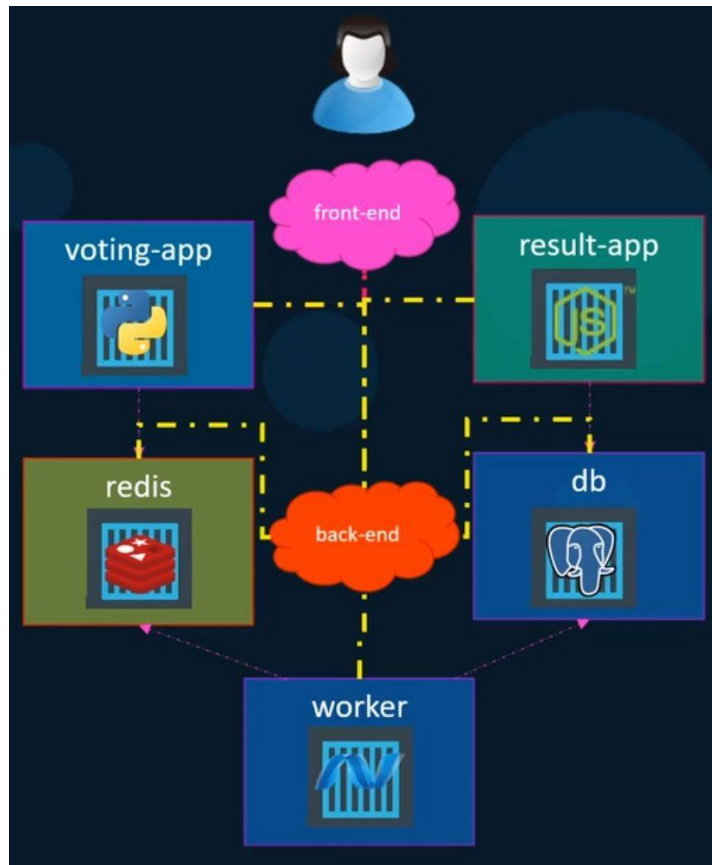
    networks:
      - back-end

  db:
    image: postgres:9.4
    networks:
      - back-end

  vote:
    image: voting-app
    networks:
      - front-end
      - back-end

  result:
    image: result
    networks:
      - front-end
      - back-end

networks:
  front-end:
  back-end:
```



Docker Compose: Voting App (Cont.)

Lets run the voting app and check its docker-compose.yml file!

- <https://github.com/dockersamples/example-voting-app>